

Rapport SAE 3.02 : Conception et développement d'une architecture multisites

Table des matières

Table des matières	1
I - Introduction.....	2
II – Fonctionnalités mises en place	2
Fonctionnalités majeures	2
Fonctionnalités mineures	3
Fonctionnalités supplémentaires (facultatives)	4
III – Difficultés rencontrées.....	4
Docker	4
Interface graphique	5
Traitement des fichiers	5
Conclusion	5
IV - Compétences développées	5
V – Diagram de GANTT	6
VI - Aspects critiques	6
VII – Conclusion	7

I - Introduction

Vous trouverez ici le rapport complet de la SAE-3.02 intitulée "*Conception et Développement d'une Architecture Multi-Serveurs pour Compilation et Exécution de Programmes*". Ce projet avait pour objectif de développer, en Python, un client capable d'envoyer des fichiers de type C, C++, Java et Python à un serveur maître via une interface graphique. Le serveur maître redistribuait ensuite ces fichiers vers plusieurs serveurs esclaves, permettant ainsi une répartition équitable des charges pour optimiser l'efficacité. Une fois un fichier reçu par un serveur esclave, celui-ci le compilait, l'exécutait, puis renvoyait la réponse au serveur maître, qui à son tour la transmettait au client. Je détaillerai dans ce rapport les aspects liés à la réalisation du projet, à son organisation, aux défis rencontrés ainsi qu'aux compétences mobilisées. Pour une analyse plus approfondie du code, je vous invite à consulter la documentation dédiée aux développeurs.

II – Fonctionnalités mises en place

Dans le cahier des charges du projet, différentes fonctionnalités ont été demandées. Dans cette partie, je vais vous présenter les fonctionnalités réalisées. Certaines ont été spécifiquement demandées, tandis que d'autres ont été ajoutées en constatant leur utilité au cours du développement.

Fonctionnalités majeures

1. **Gestion de multiples clients**

Plusieurs clients peuvent se connecter simultanément au serveur sans problème. Cela est rendu possible principalement grâce à l'attribution d'un identifiant unique à chaque client. Lorsqu'un client se connecte, le serveur maître lui assigne cet identifiant, ce qui permet de l'identifier et de lui renvoyer les résultats de ses fichiers sans risque d'erreur.

2. **Conteneurisation**

Placer les serveurs esclaves dans des conteneurs simplifie leur utilisation par le client. Ce dernier n'a qu'à exécuter une seule commande : `docker-compose up --build`. Cette commande :

- Installe les dépendances nécessaires,
- Configure les variables d'environnement,
- Alloue la mémoire (RAM) aux serveurs,
- Affiche distinctement les logs des serveurs dans le même terminal,
- Sécurise les serveurs esclaves en bloquant leur accès vers l'extérieur,
- Lance les serveurs avec la commande `python3 server.py`.

En résumé, cette solution réduit considérablement les efforts nécessaires pour déployer le projet, rendant le processus bien plus accessible.

3. Répartition des charges

Le serveur maître n'exécute aucun fichier. Son rôle est de réceptionner les fichiers envoyés par les clients et de les distribuer aux serveurs esclaves. Afin d'optimiser la répartition des charges, la distribution des fichiers se fait d'abord en fonction de leur langage. Chaque serveur esclave est priorisé pour un type de fichier : par exemple, le serveur 1 priorise les fichiers Python, tandis que le serveur 2 s'occupe des fichiers C, etc.

Cependant, cette méthode seule peut s'avérer insuffisante dans le cas d'un envoi massif de fichiers d'un même type. Pour cette raison, un second critère est pris en compte : la consommation de RAM des serveurs. Lorsqu'un fichier est reçu, le serveur maître vérifie à la fois son langage et la consommation de RAM en temps réel du serveur priorisé. Si celle-ci dépasse une limite définie dans le code, le serveur maître vérifie si un autre conteneur est disponible et respecte les seuils de consommation.

Cette répartition des charges, basée sur deux critères (langage et consommation de RAM), garantit une gestion efficace et équilibrée.

Fonctionnalités mineures

1. Validation des saisies utilisateur

J'ai utilisé des expressions régulières (regex) dans l'application client pour garantir que les informations saisies dans les champs de l'adresse IP du serveur et du port soient correctes. Cela améliore l'expérience utilisateur en évitant les erreurs liées à des entrées invalides.

2. Sélection de fichier conditionnelle

L'utilisateur ne peut sélectionner un fichier qu'après s'être connecté au serveur maître. Cette restriction prévient toute confusion, notamment l'envoi d'un fichier sans destination définie.

3. Compatibilité des fichiers

Lorsqu'un utilisateur sélectionne un fichier à envoyer au serveur, si ce fichier n'est pas de type Python, C, C++, ou Java, un message d'erreur s'affiche pour informer que le format du fichier n'est pas pris en charge. Cela garantit que seuls les fichiers compatibles peuvent être envoyés.

4. Connexion sécurisée

Pour accéder à la page `index.py`, qui permet d'interagir avec les serveurs, l'utilisateur doit d'abord s'authentifier sur la page `connexion.py`. Sans cette étape, l'accès à `index.py` est impossible, renforçant ainsi la sécurité de l'application.

5. Aide intégrée

Un bouton "Aide ?" est présent sur l'interface graphique du client pour accéder rapidement aux informations nécessaires à l'utilisation de l'application. Cela évite de devoir consulter la documentation client pour comprendre son fonctionnement, rendant l'utilisation plus intuitive.

Fonctionnalités supplémentaires (facultatives)

1. Monitoring du cluster

La visualisation de l'état des conteneurs s'effectue via le serveur maître, qui affiche régulièrement la consommation de RAM de chaque conteneur esclave. L'utilisation de Docker ouvre également la porte à des solutions de monitoring plus avancées, comme la commande `docker stats` ou les services web proposés par Docker. Ces outils nécessitent des configurations supplémentaires, mais une fois mis en place, ils permettent un suivi encore plus détaillé et performant.

2. Sécurité

Comme mentionné précédemment, une authentification est requise pour interagir avec les serveurs via l'application. De plus, l'utilisation de conteneurs apporte une couche de sécurité supplémentaire pour la machine hôte. Par exemple, si un fichier malveillant est exécuté sur un serveur esclave, son impact reste limité à l'intérieur du conteneur concerné. Cependant, aucune connexion sécurisée de type SSL n'a été implémentée dans ce projet.

3. Persistance des données

Non réalisée.

4. Scalabilité

Non réalisée. La scalabilité n'a pas pu être mise en œuvre dans ce projet, car la création automatique de nouveaux conteneurs en cas de surcharge des serveurs esclaves est particulièrement complexe. Cela nécessiterait de générer un nouveau fichier `Dockerfile`, de supprimer les conteneurs existants, puis de tout réinstaller. Ce processus prendrait plusieurs minutes, retardant ainsi le traitement des demandes des clients.

5. Robustesse

Non réalisée.

III – Difficultés rencontrées

La réflexion et la création du projet n'ont pas toujours été évidentes. Dans cette partie, je vais vous exposer les points qui m'ont pris le plus de temps.

Docker

La création des conteneurs a été, pour plusieurs raisons, l'une des parties les plus longue du projet. Étant donné que c'était la première fois que je travaillais avec un `Dockerfile` ou un `docker-compose`, j'ai rencontré quelques difficultés. Initialement, j'avais envisagé de placer le serveur maître dans un conteneur également. Après plusieurs heures de travail, je me suis rendu compte qu'il était impossible de récupérer la mémoire des autres conteneurs depuis le conteneur du serveur maître.

Léonard Sero RT22

J'ai exploré plusieurs solutions, mais j'ai finalement dû admettre qu'il me faudrait revoir le docker-compose et faire le choix de placer le serveur maître en dehors d'un conteneur. Au final, bien que la mise en place de Docker m'ait pris quelques jours, je considère que cette approche a simplifié l'installation et le démarrage des serveurs esclaves, comme expliqué précédemment. Avec du recul, je pense que le temps investi sur cette partie m'a permis d'éviter des complications ailleurs. De plus, cela m'a donné l'opportunité d'avoir une première approche de Docker dans un projet universitaire avec un cas concret.

Interface graphique

Au départ, j'avais conçu une interface graphique trop complexe, avec un nombre excessif de pages, ce qui m'a beaucoup retardé. J'ai voulu rendre l'application trop compliquée, par exemple en utilisant un fichier JSON pour enregistrer les serveurs que l'utilisateur souhaitait conserver. Après avoir passé plusieurs heures à résoudre des conflits, j'ai pris la décision de repartir de zéro pour l'interface graphique. Bien que cela ait retardé le projet, j'ai finalement réussi à la finaliser assez rapidement.

Traitement des fichiers

J'ai également passé une grande partie de mon temps à gérer les fichiers. Dans un premier temps, je stockais les fichiers sur le serveur maître, puis une fois envoyés au serveur esclave, ils étaient à nouveau enregistrés, exécutés, et renvoyés au maître pour être réenregistrés. Cette approche a engendré de nombreux problèmes. J'ai finalement décidé de revoir entièrement la gestion des fichiers. Désormais, le serveur maître stocke simplement l'ID du client, le nom du fichier et son contenu dans une variable, ce qui simplifie la transmission des fichiers et évite les conflits.

Conclusion

Ces trois points ont été les principaux obstacles qui ont ralenti l'avancement du projet. On peut en conclure que ce qui m'a le plus retardé a été le manque de réflexion en amont pour certaines tâches. Je ne me suis pas suffisamment préoccupé des aspects techniques et des contraintes avant de commencer, ce qui a été le cas pour l'interface graphique et le traitement des fichiers. C'est une erreur dont j'ai pris conscience et que je ne reproduirai pas dans mes futurs projets.

IV - Compétences développées

Voici les compétences que j'ai pu développer tout au long de cette SAE :

- **Anglais** : J'ai consulté divers forums en anglais pour trouver des solutions à mes problèmes liés à la gestion de la RAM dans les conteneurs.
- **Python** : C'est le langage principal utilisé pour ce projet.
- **PyQt** : Bibliothèque Python utilisée pour développer l'interface graphique. J'avais déjà eu l'occasion de travailler avec d'autres bibliothèques Python pour créer des interfaces graphiques, telle que Kivy.

- **Gestion de projet** : Organisation des tâches à l'aide d'un diagramme de Gantt, ce qui m'a permis de structurer efficacement les différentes étapes du projet et de respecter les délais.

V – Diagram de GANTT

Voici le diagramme de Gantt que j'ai réalisé le premier jour de mon projet. Je l'ai globalement bien suivi, à quelques ajustements près. En effet, j'ai modifier certains aspects, comme l'interface graphique, plus tard au cours du projet. La qualité de ce document n'est pas optimale, je vous invite donc à consulter le dépôt GitHub de mon projet, où j'ai publié une version PDF plus lisible.



VI - Aspects critiques

Bien entendu, ce projet peut encore être perfectionné. Voici, selon moi, ce qui pourrait être amélioré à partir de ce que j'ai déjà réalisé :

- **Sécurité** : Ajouter une connexion sécurisée SSL entre les clients et le serveur maître afin de garantir la confidentialité et l'intégrité des échanges.
- **Filtrage des fichiers** : Si un client envoie un fichier qui ne se termine jamais ou qui reste bloqué (par exemple, avec une boucle while True qui ne se termine jamais), le serveur pourrait fermer

la connexion après un certain délai, en indiquant que le fichier est irrecevable en raison de son temps d'exécution trop long.

- **Gestion des dépendances** : Si un utilisateur envoie un fichier nécessitant l'installation de dépendances, le serveur pourrait alors installer ces dépendances avant d'exécuter le code. Toutefois, cela soulève des questions de sécurité pour éviter l'installation de dépendances non fiables. Une solution pourrait être d'envoyer ces fichiers vers un conteneur dédié uniquement à ce type de tâches, afin de sécuriser l'environnement d'exécution.
- **Détection de panne** : Si l'un des conteneurs tombe en panne, le serveur maître pourrait détecter cette panne et éviter d'envoyer un fichier vers ce conteneur. Au lieu de cela, il enverrait le fichier vers un autre serveur esclave en bon état de fonctionnement.

Je trouve ces points particulièrement intéressants à développer sur la structure déjà en place, et ils pourraient nettement améliorer la valeur du projet.

VII – Conclusion

Dans ce document, je vous ai présenté comment s'est déroulée la SAE pour moi, le raisonnement qui m'a permis de mener à bien le projet, mes erreurs, les améliorations que j'y ai apportées, les fonctionnalités développées, ce qui pourrait être développé davantage, mes difficultés, etc. J'estime avoir consacré entre 50 et 55 heures à cette SAE, principalement en raison du choix d'utiliser Docker, ce qui m'a causé de nombreux problèmes. Cependant, au final, je suis satisfait du résultat et d'avoir réussi à mener le projet à bien.

J'ai trouvé cette SAE originale et j'ai particulièrement apprécié la partie sur le load balancing. Au début, cela m'a paru très compliqué et je n'étais pas très enthousiaste à l'idée de réaliser ce projet. Cependant, une fois lancé, j'ai rapidement pris plaisir à développer les fonctionnalités de mes serveurs.